AI Engineer – Personalized Educational Content Recommendation System

---

Architecture

Services :

ollama → ollama-pull → data-init → backend → frontend

Container	Role	Port
ollama	**Runs Ollama LLM server**	**11434**
ollama-pull	**Pulls phi3 model once, then exits**	**—**
data-init	**Runs llm_gen.py if sample_data.csv missing, then exits**	**—**
backend	**FastAPI REST API**	**8000**
frontend	**Streamlit UI**	**8501**

Request Flow :

```
User (browser :8501)
 │  enters topic + optional filters
 ▼
[frontend / app.py]
 │  POST /recommend → http://backend:8000
 ▼
[backend / recommender.py]
 │
 ├─ 1. Hard filter
 │     • Remove already-viewed course IDs
 │     • Apply difficulty / duration / learning style constraints
 │
 ├─ 2. Semantic retrieval
 │     • Encode user topic with all-MiniLM-L6-v2
```

```
│     • Cosine similarity against pre-embedded course titles.
│     • Pick Top-10 results
│
└─ 3. LLM re-ranking (→ http://ollama:11434)
     • Send Top-10 titles to phi3
     • phi3 re-ranks → returns Top-3 as JSON
     • Each recommendation gets a one-sentence explanation
▼
Response → frontend displays Top-3 cards
User clicks "Mark Viewed" → profiles.json updated (persisted on shared volume)
```

Storage :

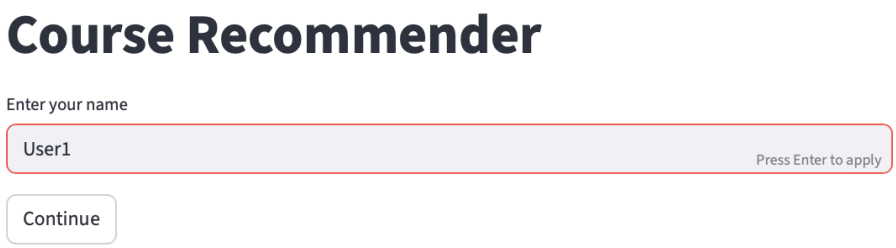
Volume	What's stored
ollama_data	**Downloaded phi3 model weights (~2 GB)**
app_data	**sample_data.csv +** **profiles.json (viewed history)**

LLM Prompt :

```
You are a course recommender. The user wants to learn: '{topic}'.

Rank the TOP 3 most relevant courses from the list below and explain why each fits.
Return ONLY a JSON array of exactly 3 objects. No markdown, no extra text.

Courses:
{json.dumps([{"id": c["id"], "title": c["title"]} for c in top10], indent=2)}

Return format:
[{{"id": 5, "explanation": "one sentence why this fits"}}, ...]"""
```

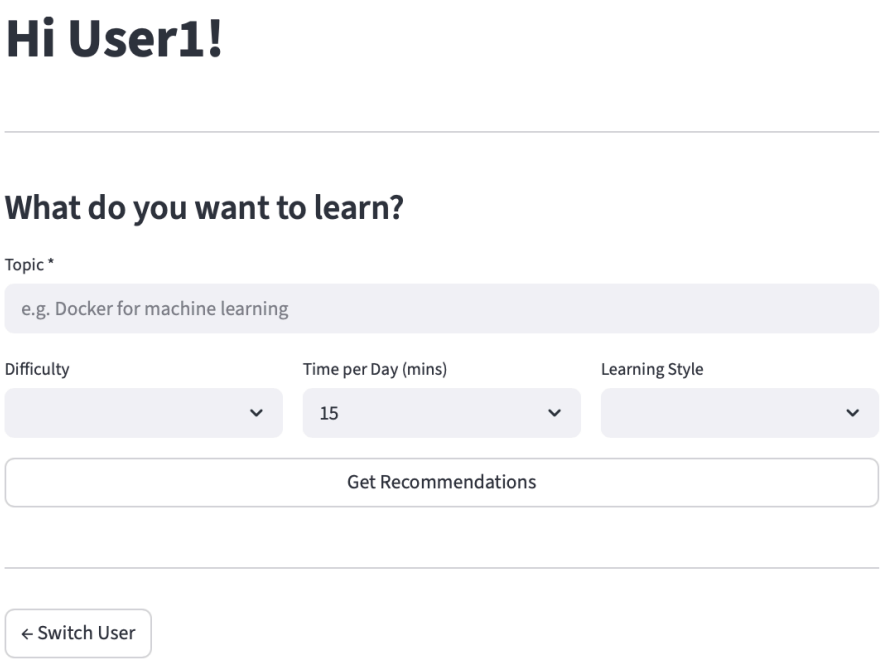
Since we are running the LLM locally, we have limitations and can only use lightweight models. Currently, we are using Microsoft's Phi-3 model, which is suitable for running locally. However, we also face limitations with complex queries. During testing, the model tends to hallucinate and sometimes fails to return results in the format we required. Therefore, LLM prompts are kept simple and clean for this purpose.

User page : User profile is created for each unique profile name.



Personalized education recommender : Recommendation page.

Here, the user can see the courses the user has already completed. And users can search for courses under the ‘Topic’ session. Difficulty/Time per day/learning style are optional filters users can choose.



Sample search :

Hi User1!

Previously viewed: • Advanced Kubernetes Deployment Strategies for High Availability and Scalability

---

What do you want to learn?

Topic *

Mastering Docker Swarm Orchestration for High Availability Production Environments

Difficulty	Time per Day (mins)	Learning Style
	90	article

Get Recommendations

Finding best courses...

Search Result :

What do you want to learn?

Topic *

Mastering Docker Swarm Orchestration for High Availability Production Environments

Difficulty	Time per Day (mins)	Learning Style
	90	article

Get Recommendations

Top Recommendations

1. Mastering Docker Swarm for High Availability Production Environments

Mark Viewed

ARTICLE | Advanced | ⏱ 90 mins

Directly matches the user's interest in high availability production environments specifically with Docker.

Open Course ↗

2. Mastering Docker Swarm Orchestration for High Availability Environments

Mark Viewed

ARTICLE | Advanced | ⏱ 15 mins

Focuses on orchestration and is relevant to achieving high availability but does not specify production environments.

Open Course ↗

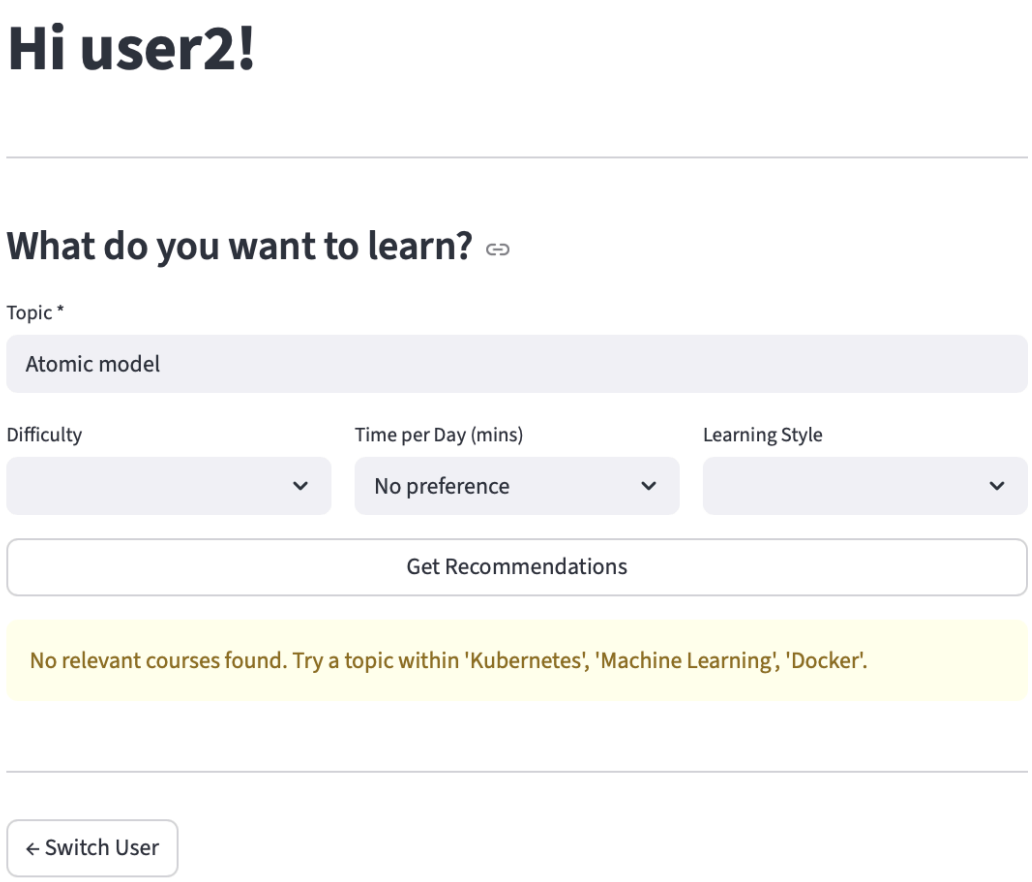
3. Mastering Kubernetes for High Availability Production Environments

Mark Viewed

ARTICLE | Advanced | ⏱ 45 mins

Although about Docker Swarm's close relative Kubernetes and tailored to high availability in production environments.

Open Course ↗

Additional Case :

If no relevant results (in terms of cosine similarity), No results will be returned.